

GPU-Accelerated 2D LIDAR SLAM with Scan-to-Map ICP

基于 CUDA 的二维激光雷达 SLAM：扫描-地图 ICP 配准的并行实现

A code-level technical report on `slam.cu` | 关于 `slam.cu` 的代码级技术报告

Abstract 摘要

This report documents a compact CUDA implementation of two-dimensional LIDAR Simultaneous Localisation and Mapping (SLAM). The system estimates a sequence of robot poses by aligning each incoming laser scan against an incrementally accumulated point map using Iterative Closest Point (ICP). Two stages are offloaded to the GPU: the rigid transformation of scan points under a candidate pose, and the brute-force nearest-neighbour search that establishes point correspondences. The remaining stages - file parsing, the closed-form two-dimensional Kabsch rotation fit, and the outer iteration loops - remain on the host because their operand counts are small and their control flow is inherently sequential. We describe the algorithm, derive the closed-form pose solver, analyse the asymptotic cost of the dominant kernel, and identify the design limits that would need to be addressed before the system could run on long trajectories.

中文：本报告记录了一个精简的二维激光雷达 SLAM（同步定位与建图）系统的 CUDA 实现。该系统采用 ICP（迭代最近点）算法，将每一帧新到达的激光扫描与逐步累积的点云地图进行配准，从而估计机器人的位姿序列。其中两个阶段被卸载到 GPU 上执行：一是在候选位姿下对扫描点做刚体变换，二是用暴力搜索为每个扫描点在地图中寻找最近邻，以建立点对应关系。其余部分——文件读取、二维 Kabsch 旋转的闭式求解、以及外层迭代循环——仍然保留在主机（CPU）端，因为它们的数据量很小且控制流本质上是串行的。本文将阐述算法流程、推导闭式位姿求解公式、分析主导核函数的渐近复杂度，并指出该系统若要处理长轨迹所必须解决的设计瓶颈。

1. Introduction 引言

SLAM asks a robot to answer two coupled questions at once: where am I, and what does the world around me look like? The two questions are coupled because a map is only meaningful relative to the poses from which it was observed, and a pose is only estimable relative to a map. The classical way to break the circularity is to alternate: freeze the map, estimate the newest pose against it, then freeze that pose and fold the new observation into the map. The implementation studied here follows exactly this alternation, with ICP as the pose estimator.

中文：SLAM 要求机器人同时回答两个互相纠缠的问题：我在哪里？周围的世界长什么样？这两个问题之所以纠缠，是因为地图只有相对于观测它的位姿才有意义，而位姿也只有相对于地图才能被估计。打破这种循环依赖的经典做法是“交替固定”：先固定地图，用它来估计最新的位姿；再固定该位姿，把新的观测融入地图。本文研究的实现正是遵循这一交替过程，并用 ICP 作为位姿估计器。

The interesting engineering question is not whether ICP works - it is well established - but which parts of it deserve a GPU. The answer in this code is driven by a single observation: correspondence search is the only stage whose cost grows without bound as the mission proceeds, because the map it searches grows with every scan. Everything else operates on a fixed, small number of points. The partition of labour between host and device follows directly from that asymmetry.

中文：真正有趣的工程问题不是 ICP 是否有效——这早已被验证——而是它的哪些部分值得放到 GPU 上。本代码给出的答案源于一个观察：只有“对应点搜索”这一阶段的开销会随任务推进而无限增长，因为它所搜索的地图每来一帧扫描就变大一次；其余步骤处理的点数始终是固定且很小的。主机与设备之间的分工正是直接由这种不对称性决定的。

2. System Overview 系统总览

The program consumes a directory of CSV files: a manifest giving the scan count and the nominal points-per-scan, followed by one file per scan holding raw (x, y) points in the sensor frame. It produces a single trajectory file of (x, y, θ) triples. The pipeline per scan is shown below.

中文：程序读取一个 CSV 目录：manifest

文件给出扫描帧数与每帧的标称点数，随后每一帧对应一个文件，存放传感器坐标系下的原始 (x, y) 点。程序最终输出一个包含 (x, y, theta) 三元组的轨迹文件。每帧的处理流程如下表。

Stage 阶段	Where 位置	Cost 开销	Rationale 理由
CSV parse 读取扫描	Host 主机	$O(n)$	I/O bound; no arithmetic to parallelise. 受 I/O 限制，无可并行的计算。
Constant-velocity guess 匀速运动预测	Host 主机	$O(1)$	Three subtractions per scan. 每帧仅三次减法。
Transform points 点云刚体变换	Device 设备	$O(n)$	Perfectly data-parallel, one thread per point. 完全数据并行，一点一线程。
Nearest neighbour 最近邻搜索	Device 设备	$O(n \times m)$	Dominant cost; m grows every frame. 主导开销，m 每帧增长。
Gather correspondences 筛选对应点	Host 主机	$O(n)$	Requires a device-to-host copy first. 需先做设备到主机的拷贝。
Kabsch pose fit Kabsch 位姿求解	Host 主机	$O(k)$	$k \leq n$; four sums and one atan2. k 不超过 n，仅四个求和与一次 atan2。
Append to map 并入地图	Both 两者	$O(n)$	Transform on GPU, mirror kept on host. GPU 上变换，主机保留镜像副本。

Table 1. Per-scan pipeline. n = points in the current scan, m = points already in the map. 表 1：单帧处理流程； n 为当前扫描点数， m 为地图已有点数。

3. The GPU Kernels GPU 核函数

3.1 transformKernel - rigid transformation 刚体变换核

A candidate pose (x, y, theta) maps a point p in the sensor frame to the world frame by a rotation followed by a translation. Each output point depends only on its own input, so the mapping is embarrassingly parallel: one thread per point, no synchronisation, no shared memory, coalesced reads and writes because thread i touches element i .

中文：候选位姿 (x, y, theta) 通过"先旋转、后平移"把传感器坐标系下的点 p 映射到世界坐标系。每个输出点只依赖于它自己的输入，因此这是典型的"易并行"问题：一点一线程，无需同步、无需共享内存；由于第 i 个线程只访问第 i 个元素，读写天然合并 (coalesced)。

```
__global__ void transformKernel(const float2 *in, int n, Pose pose, float2 *out) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;
    float c = cosf(pose.theta), s = sinf(pose.theta);
    out[i].x = c * in[i].x - s * in[i].y + pose.x;
    out[i].y = s * in[i].x + c * in[i].y + pose.y;
}
```

Listing 1. Rotation and translation, one thread per point. 清单 1：一点一线程的旋转与平移。

Note that `cosf` and `sinf` are recomputed identically by every thread. This is redundant but harmless: the two transcendentals are cheap on the special function unit relative to the global memory traffic, and passing them as pre-computed arguments would save little. The guard `if (i >= n) return;` is required because the launch rounds the thread count up to a multiple of the block size (128).

中文：注意每个线程都在重复计算完全相同的 `cosf` 与 `sinf`。这虽有冗余但无害：相对于全局内存访问，这两个超越函数在特殊功能单元上开销极低，即便预先算好再作为参数传入也节省不了多少。而 `if (i >= n) return;` 这一守卫是必需的，因为启动时线程数被向上取整为块大小 (128) 的整数倍。

3.2 nearestNeighborKernel - correspondence search 最近邻搜索核

This is the heart of the system and the reason a GPU is present at all. For each of n transformed scan points, the kernel scans all m map points and records the index and squared distance of the closest one. Squared distance is used throughout - the square root is monotonic, so omitting it changes nothing about which point wins while removing one expensive operation from the innermost loop.

中文：这是整个系统的核心，也是引入 GPU 的根本原因。对于 n 个已变换的扫描点，核函数逐一遍历全部 m 个地图点，记录距离最近者的索引与平方距离。全程使用平方距离——因为开方是单调函数，省略它不会改变谁是最近点，却能从最内层循环中去掉一次昂贵运算。

```
__global__ void nearestNeighborKernel(const float2 *query, int nq,
                                     const float2 *target, int nt,
                                     int *bestIdx, float *bestDist2) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= nq) return;
    float qx = query[i].x, qy = query[i].y;
    float best = FLT_MAX; int besti = -1;
    for (int j = 0; j < nt; ++j) {
        float dx = qx - target[j].x, dy = qy - target[j].y;
        float d2 = dx * dx + dy * dy;
        if (d2 < best) { best = d2; besti = j; }
    }
    bestIdx[i] = besti; bestDist2[i] = best;
}
```

Listing 2. Brute-force nearest neighbour, parallelised over query points. 清单 2：在查询点维度上并行的暴力最近邻搜索。

The parallelisation is over queries, not over targets. Every thread walks the entire map independently, which means the total arithmetic is unchanged from the serial algorithm - only the wall-clock time is divided. The access pattern is favourable in one respect and wasteful in another: all threads in a warp read the same map element j at the same step, so the read broadcasts efficiently, but the same map data is re-read from global memory by every block, scan after scan, iteration after iteration.

中文：并行的维度是“查询点”而非“目标点”。每个线程都独立遍历整张地图，这意味着总运算量与串行算法完全相同，被切分的只是墙钟时间。这种访存模式一方面有利：一个 warp 内的所有线程在同一步读取同一个地图元素 j ，因此该读取可以高效广播；另一方面又很浪费：同一份地图数据被每个线程块、每一次迭代、每一帧扫描反复从全局内存中重新读取。

4. Host-Side Pose Solving 主机端位姿求解

Given k correspondences between source points p (in the sensor frame) and target points q (in the map frame), the optimal rigid transform minimising the sum of squared residuals has a closed form. Centre both sets on their centroids, accumulate the four cross-covariance sums, and read the rotation off an arctangent:

中文：给定 k 组对应关系——源点 p （传感器坐标系）与目标点 q （地图坐标系）——使残差平方和最小的最优刚体变换存在闭式解。做法是：先把两组点各自中心化到质心，再累加四个互协方差和，最后用一次反正切即可读出旋转角：

```
theta = atan2( Sxy - Syx ,  Sxx + Syy )
t      = centroid(q) - R(theta) * centroid(p)

where  Sxx = sum(px*qx),  Sxy = sum(px*qy),
       Syx = sum(py*qx),  Syy = sum(py*qy)
       (p, q centred on their respective centroids)
```

Listing 3. Closed-form 2D Kabsch solution. 清单 3：二维 Kabsch 闭式解。

This is the two-dimensional specialisation of the Kabsch/Horn algorithm. It is equivalent to an SVD of the cross-covariance matrix, but in the plane the SVD collapses to a single arctangent - a fact most easily seen by treating each point as a complex number, where the optimal rotation is simply the argument of the sum of $\text{conj}(p) * q$. The accumulators are declared `double` even though the point data is `float`, which guards against cancellation when the centred coordinates are small relative to their spread.

中文：这是 Kabsch/Horn 算法在二维情形下的特化。它等价于对互协方差矩阵做 SVD，但在平面上 SVD 退化为一次反正切——把每个点视为复数即可最直观地看出这一点：此时最优旋转角就是 $\text{sum}(\text{conj}(p) * q)$ 的辐角。值得注意的是，尽管点数据是 float，累加器却声明为 double，这可以防止在中心化坐标数值远小于其分布范围时出现抵消误差。

Crucially, the solver is fed the original local-frame points as source, not the transformed ones. Each ICP iteration therefore re-estimates the pose from scratch rather than composing an incremental correction onto the previous estimate. Both formulations are valid; this one avoids accumulating composition error across the eight iterations.

中文：一个关键细节：求解器的源点用的是"原始"局部坐标系下的点，而不是已变换的点。因此每一次 ICP 迭代都是从头重新估计位姿，而非在上一次估计之上叠加一个增量修正。两种写法都成立，但这种写法可以避免在八次迭代中累积复合变换的误差。

5. The ICP Loop and Motion Prediction ICP 循环与运动预测

Scan 0 defines the origin: its pose is the identity, so the entire estimated trajectory lives in a frame anchored wherever the robot happened to start. For every subsequent scan the loop performs eight iterations of predict, associate, solve.

中文：第 0 帧定义原点：其位姿被设为单位变换，因此整条估计轨迹都处于一个"以机器人起始位置为锚点"的坐标系中。此后每一帧都会执行八次"预测—关联—求解"的迭代循环。

```
guess.x      = pose.x      + (pose.x      - prevPose.x);    // constant velocity
guess.y      = pose.y      + (pose.y      - prevPose.y);
guess.theta  = pose.theta  + (pose.theta  - prevPose.theta);

for (iter = 0; iter < 8; ++iter) {
    transformKernel    <<<blocks,128>>>(d_local, n, iterPose, d_transformed);
    nearestNeighborKernel<<<blocks,128>>>(d_transformed, n, d_map, mapCount, ...);
    // copy indices + distances back, keep pairs with d2 < 0.36
    if (src.size() < 5) break;                // too little overlap: keep last pose
    iterPose = solveRigidTransform(src, dst);
}
```

Listing 4. Outer ICP iteration. 清单 4：外层 ICP 迭代。

Two robustness devices are worth naming. The constant-velocity prediction extrapolates the previous inter-frame motion instead of assuming the robot stood still, which places the initial guess much closer to the true pose and keeps ICP inside its basin of convergence during sustained motion. The distance gate (0.6 m, compared in squared form against 0.36) rejects correspondences that are almost certainly spurious - newly observed geometry with no counterpart in the map, or gross mismatches early in the iteration.

中文：有两处鲁棒性设计值得点名。其一是匀速运动预测：它外推上一帧的帧间运动，而不是假设机器人静止不动，从而让初始猜测更接近真实位姿，使 ICP

在机器人持续运动时仍能停留在收敛域内。其二是距离门限（0.6 米，以平方形式与 0.36 比较）：它剔除那些几乎必然错误的对应关系——比如地图中尚无对应物的新观测几何，或迭代早期出现的严重误匹配。

A subtle consequence of the early **break**: when fewer than five gated correspondences survive, the loop exits and **iterPose** retains whatever value it last held. On the first iteration that value is the constant-velocity guess, so the system degrades to dead reckoning rather than failing outright - a reasonable fallback, though one that will drift silently.

中文：提前 break 还有一个微妙的后果：当通过门限的对应点少于五个时，循环退出，iterPose 保持其最后一次的取值。若这发生在第一次迭代，该值就是匀速预测的猜测位姿，于是系统退化为航位推算（dead reckoning），而不是直接失败。这是一个合理的兜底策略，但它会在无声无息中产生漂移。

6. Complexity and Memory 复杂度与内存

Let N be the number of scans and n the points per scan. After scan i the map holds $i \cdot n$ points, so the nearest-neighbour work for scan i is $8 \cdot n \cdot (i \cdot n)$ comparisons across the eight ICP iterations. Summing over the run:

中文：设 N 为扫描帧数， n 为每帧点数。处理完第 i 帧后地图中共有 $i \cdot n$ 个点，因此第 i 帧在八次 ICP 迭代中的最近邻工作量为 $8 \cdot n \cdot (i \cdot n)$ 次比较。对全程求和得：

```
total comparisons = 8 * n^2 * sum(i, i=1..N-1)  ~=  4 * n^2 * N^2
e.g. n = 90, N = 200  ->  ~1.3 x 10^9 comparisons
```

Listing 5. Quadratic growth in both scan count and scan density. 清单 5：比较次数随帧数与点密度双双呈平方增长。

The cost is quadratic in the number of scans. This is the defining limitation of the design: it is entirely comfortable for a few hundred scans and entirely impractical for a few thousand, regardless of how fast the GPU is. Parallelism lowers the constant factor; it does not change the exponent.

中文：开销与帧数呈平方关系。这正是该设计的根本局限：处理几百帧时游刃有余，处理几千帧时则完全不切实际——无论 GPU 有多快。并行化只能降低常数因子，改变不了指数。

Memory is handled by a single up-front allocation strategy. All device buffers are sized once for the worst case before the loop begins, so no `cudaMalloc` ever appears on the hot path. The map buffer is allocated for $N \cdot n$ points, the theoretical maximum. A host-side mirror of the map is also maintained, because correspondence targets must be gathered on the CPU for the Kabsch fit; this doubles the memory footprint but avoids a device-to-host copy of the full map every iteration.

中文：内存采用“一次性预分配”策略。所有设备端缓冲区都在循环开始前按最坏情况一次性分配，因此热路径上不会出现任何 `cudaMalloc`。地图缓冲区按理论最大值 $N \cdot n$ 个点分配。同时还维护了一份地图的主机端镜像，因为 Kabsch 求解所需的对应目标点必须在 CPU 上收集；这使内存占用翻倍，但避免了每次迭代都把整张地图从设备拷回主机。

7. Limitations and Future Work 局限与改进方向

The following are the honest weak points of the implementation, roughly in order of how much they would repay attention.

中文：以下是该实现诚实的薄弱之处，大致按“改进后收益从大到小”排序。

- Unbounded map growth. Every scan is appended in full, with no voxel filter or duplicate removal. Re-observing the same wall a hundred times stores it a hundred times. A voxel grid of 5-10 cm would cap the map size in a bounded environment and turn quadratic growth into linear.
- Brute-force search. A uniform spatial hash or a k-d tree would reduce per-query cost from $O(m)$ to roughly $O(1)$ or $O(\log m)$. A grid is the natural choice here: the distance gate already bounds the search radius to 0.6 m, so only the neighbouring cells need visiting.
- Round-trip per iteration. Eight device-to-host copies per scan serialise CPU and GPU. The correspondence gather and the four covariance sums are reductions and could stay on the device, leaving only a handful of scalars to transfer.
- No loop closure. Errors accumulate monotonically; revisiting a known place does not correct the trajectory. Pose-graph optimisation is the standard remedy and the single largest accuracy improvement available.
- Point-to-point metric. A point-to-plane error, using local surface normals, converges in fewer iterations and slides more gracefully along featureless walls - a common failure mode in corridors.
- Timing scope. The CUDA events bracket the entire loop including CSV parsing, so the reported figure is wall-clock time, not isolated GPU time. Per-kernel events would be more informative.
- 地图无界增长。每帧都被完整追加，既无体素滤波也无重复点剔除。同一面墙被观测一百次就会被存储一百次。采用 5~10 厘米的体素栅格可在有界环境中限制地图规模，把平方增长变为线性增长。
- 暴力搜索。使用均匀空间哈希或 k-d 树可把单次查询的开销从 $O(m)$ 降到约 $O(1)$ 或 $O(\log m)$ 。此处栅格是最自然的选择：距离门限已经把搜索半径限定在 0.6 米内，只需访问相邻的少数格子。
- 每次迭代的往返拷贝。每帧八次的设备到主机拷贝使 CPU 与 GPU 相互串行等待。对应点收集与四个协方差求和本质上都是归约操作，完全可以留在设备端完成，最终只需传回几个标量。

- 没有回环检测。误差单调累积，重访已知地点也无法修正轨迹。位姿图优化是标准解法，也是目前可获得的最大精度提升。
- 点到点度量。改用基于局部法向量的点到面误差，可以用更少的迭代收敛，并且在无特征的墙面上滑动得更自然——这正是走廊场景中的常见失效模式。
- 计时范围。CUDA event 包裹了包含 CSV 解析在内的整个循环，因此报告的数值是墙钟时间而非纯 GPU 时间。逐核函数打点会更有参考价值。

8. Building and Running 编译与运行

```
nvcc -O2 -arch=sm_70 src/slam.cu -o slam      # sm_60 Pascal / sm_86 Ampere
python generate_test_data.py                  # populates data/
./slam data                                  # -> data/estimated_traj.csv
python plot_results.py                        # compare against ground truth
```

Listing 6. Build and execution. 清单 6：编译与执行流程。

One caveat when interpreting the output: the estimated trajectory is expressed in the SLAM frame, which is the world frame rotated and translated by wherever scan 0 happened to be. A direct numerical comparison against ground truth is therefore meaningless until the two are aligned by a single global rigid transform.

中文：解读输出时有一点需注意：估计出的轨迹表达在 SLAM

自身的坐标系中，该坐标系相当于世界坐标系被平移旋转到第 0

帧扫描所在的位置。因此在用一个全局刚体变换把两者对齐之前，直接把它与真值做数值比较是没有意义的。

9. Conclusion 结论

The implementation is a clear demonstration of a principle that matters more than any individual optimisation: profile the asymptotics before writing kernels. Only the two stages whose work scales with the map were moved to the GPU, and the rest was deliberately left on the host where its sequential nature costs nothing. The result is a system that is easy to read and correct in its scope. Its ceiling is set not by the quality of the CUDA code but by the choice of a brute-force search and an ever-growing map - and both of those are algorithmic choices, fixable without touching a single thread index.

中文：这份实现清晰地展示了一个比任何单项优化都更重要的原则：先分析渐近复杂度，再动手写核函数。只有工作量随地图规模增长的那两个阶段被移到 GPU，其余部分则被有意留在主机端——在那里，它们的串行本质并不带来任何代价。最终得到的系统易读，且在其适用范围内是正确的。它的性能天花板并非由 CUDA 代码的质量决定，而是由“暴力搜索”与“无限增长的地图”这两个选择决定——而这两者都属于算法层面的选择，无需改动任何一个线程索引即可修正。